RESEARCH PROJECT

Liquid Neural Networks for AI-Based CPU Scheduling: An Empirical Study of Burst Prediction

Farhan Rashid | 24BAI0303

CfC versus LSTM on synthetic arrivals and Alibaba PAI job submissions

1. Introduction

Research question: does explicit elapsed time help predict future arrival bursts?

Scheduling motivation

A warning of rising job arrivals could inform future resource allocation and scheduling decisions.

Continuous-time prediction

CfC receives irregular arrivals with elapsed time. The LSTM baseline receives counts in fixed time bins.

Implemented scope

This study trains burst classifiers. It does not implement a CPU scheduler or measure waiting-time improvements.

Evidence

Three datasets, two parameter-matched models, held-out classification metrics and CPU inference timing.

Source: local project implementation and saved experiment outputs

2. Detailed Literature Review — Overview

40 papers reviewed across four themes (IEEE, Springer, ACM, Google Scholar)

A

DRL-Based CPU / Cluster Scheduling — 14 papers

DeepRM, A2C schedulers, hierarchical DRL, HPC/energy-aware RL scheduling — the discrete-time baseline this project compares against.

B

Edge Computing & Heterogeneous Hardware — 4 papers

DRL scheduling on heterogeneous multi-core, edge GPU, and PIM architectures.

C

Liquid Neural Networks / CfC Networks — 15 papers

Core LNN/CfC theory plus applications in forecasting, robotics, healthcare, and manufacturing — including the closest prior art to this project.

D

Genetic / Evolutionary Scheduling Algorithms — 7 papers

Non-ML metaheuristic scheduling — the classical contrast class.

Literature Review — A. DRL-Based Scheduling (1/2)

1. **Resource Management with Deep Reinforcement Learning (DeepRM)** *Mao, Alizadeh, Menache, Kandula*
2. **Deep Reinforcement Learning for Multi-Resource Multi-Machine Job Scheduling**
3. **A New Approach for Resource Scheduling with Deep Reinforcement Learning**
4. **Optimizing Memory Mapping Using Deep Reinforcement Learning (mallocGame)**
5. **DeepScheduler: An Advantage Actor-Critic Based Approach for Job Scheduling**
6. **A2cScheduler: Advantage Actor-Critic Deep RL Based Approach**
7. **On the Impact of MDP Design for RL Agents in Resource Management**

Literature Review — A. DRL-Based Scheduling (2/2)

8 **Deep Reinforcement Learning with Different Rewards for Scheduling in HPC Systems**

9 **Reinforcement Learning Based Multi-Core Scheduling (RLBMCS) for Real-Time Systems** *Rinku & AshaRani*

10 **Multiple Resource Management and Burst Time Prediction using DRL**

11 **Energy-Efficient Task Scheduling in Data Centers Using Adaptive DRL (PPO)**

12 **Adaptive Energy-Aware Scheduling Policy for Grid Systems Using DRL**

13 **Self-Adapting CPU Scheduling for Mixed Database Workloads via Hierarchical DRL** *Xing, Wang, Liu — Symmetry, 2025*

14 **Deep RL for Job Scheduling and Resource Management in Cloud Computing (Review)** *Gu et al. — arXiv:2501.01007*

Literature Review — B. Edge Computing & Heterogeneous Hardware

15 **Task Scheduling for Heterogeneous Multi-Core Processors Based on DRL** *Tan, Chen, Liu*

16 **Scheduling Techniques of AI Models on Modern Heterogeneous Edge GPU — A Critical Review** *Majeed, Meribout, Sali*

17 **THERMOS: A Deep RL Based Online Scheduling Policy for DNNs on PIM Architectures** *Chen, Gu, Huang, Lin*

18 **SplitPlace: AI-Driven Online Policy for Neural Network Split Fragments**

Literature Review — C. Liquid Neural Networks / CfC (1/2)

19 **Liquid Time-Constant Networks** *Hasani, Lechner, Amini, Rus, Grosu — AAAI 2021*

20 **Closed-Form Continuous-Time Neural Networks (CfC)** *Hasani et al. — Nature Machine Intelligence, 2022*

21 **Neural Circuit Policies Enabling Auditable Autonomy** *Lechner et al. — Nature Machine Intelligence, 2020*

22 **Conditional Skip Liquid Neural Networks for Cloud Resource Prediction** *Neural Computing and Applications, 2026*

23 **Robust Flight Navigation Out of Distribution with Liquid Neural Networks** *Chahine et al. — Science Robotics, 2023*

24 **Digital Twins for Patient Care via Knowledge Graphs and CfC Networks** *arXiv:2307.04772*

25 **Exploring Liquid Neural Networks on Loihi-2** *Kaddoura et al. — IEEE Access, 2024*

26 **Liquid Fusion of Heterogeneous Representations for Salient Object Detection** *arXiv:2606.26849*

Literature Review — C. Liquid Neural Networks / CfC (2/2)

Highlighted: the paper closest to this project's core idea

27 **From C.elegans to Liquid Neural Networks: Wind Power Prediction** *TechRxiv, 2025*

28 **LiquidTAD: Efficient Temporal Action Detection via Liquid-Inspired Relaxation** *arXiv:2604.18274*

29 **MeloTune: On-Device Arousal Learning with CfC Networks** *arXiv:2604.10815*

30 **Accuracy, Memory Efficiency and Generalization: LNNs vs. RNNs** *arXiv:2510.07578, 2025*

31 **LTC-SE: Expanding LTC Networks for Scalable AI and Embedded Systems** *Bidollahkhani et al. — arXiv:2304.08691*

32 **Invertible LNN-Based Learning of Inverse Kinematics for Robotic Manipulators** *Scientific Reports, 2025*

33 **Liquid Neural Network in Smart Manufacturing: A New Opportunity** *Leng et al. — Journal of Manufacturing Systems, 2025/26 — CLOSEST PRIOR ART*

Literature Review — D. Genetic / Evolutionary Scheduling

34 **Hybrid Task Scheduling Using Genetic Algorithms and Machine Learning** *IJFMR, 2025*

35 **Task Scheduling in Cloud Computing Using Multi-Objective Evolutionary Algorithms (Survey)** *Artificial Intelligence Review, 2024*

36 **Energy and Time Efficient Task Scheduling Using a Hybrid GA-MOPSO Approach** *J. of Kufa for Math. and Computer*

37 **CloudSched-GA: An Adaptive Genetic Optimizer for Cloud Task Scheduling** *Neural Computing and Applications*

38 **A Genetic Algorithm Inspired Task Scheduling in Cloud Computing** *Agarwal & Srivastava — IEEE ICCCA, 2016*

39 **A Probabilistic Genetic Algorithm Approach to Task Scheduling in Cloud Environments** *CLEI Electronic Journal*

40 **Task Scheduling Optimization in Cloud Computing Based on Genetic Algorithms** *Computers, Materials & Continua*

3. Problem statement and study scope

Prediction target

At time t, predict whether arrivals in (t, t + W] exceed a threshold K.

Working hypothesis

Precise inter-arrival timing may help on bursty workloads. Fixed-bin aggregation may discard useful within-bin timing.

Comparison

CfC versus LSTM only. The implemented experiments do not train GRU or reinforcement-learning schedulers.

Contribution

An empirical feasibility study. The experiments do not establish a first-ever result or universal model superiority.

Source: local project implementation and saved experiment outputs

4. Objectives and implemented evidence

Workload construction

Generate Poisson and MMPP events, then load the first 20,000 time-sorted Alibaba PAI records.

Model comparison

Preserve event timing for CfC and create fixed-bin count sequences for LSTM.

Training and testing

Use chronological 70/10/20 splits and approximately 21,000 parameters per model.

Reporting

Report AUC, F1, accuracy, parameter counts and measured CPU latency. Document limits of the comparison.

Source: local project implementation and saved experiment outputs

5. Datasets and target construction

Poisson

20,000 simulated seconds

Rate: 2 arrivals / second

W = 10 seconds; K = 25

Memoryless control workload

MMPP

20,000 simulated seconds

Quiet / busy: 0.5 / 15 arrivals/s

Switch rates: 0.01 / 0.05 per s

W = 10 seconds; K = 25

Alibaba PAI

v2020 job-duration-estimate CSV

First 20,000 sorted submissions

Only submit_time supplies inputs

W = 300 seconds; K = 10

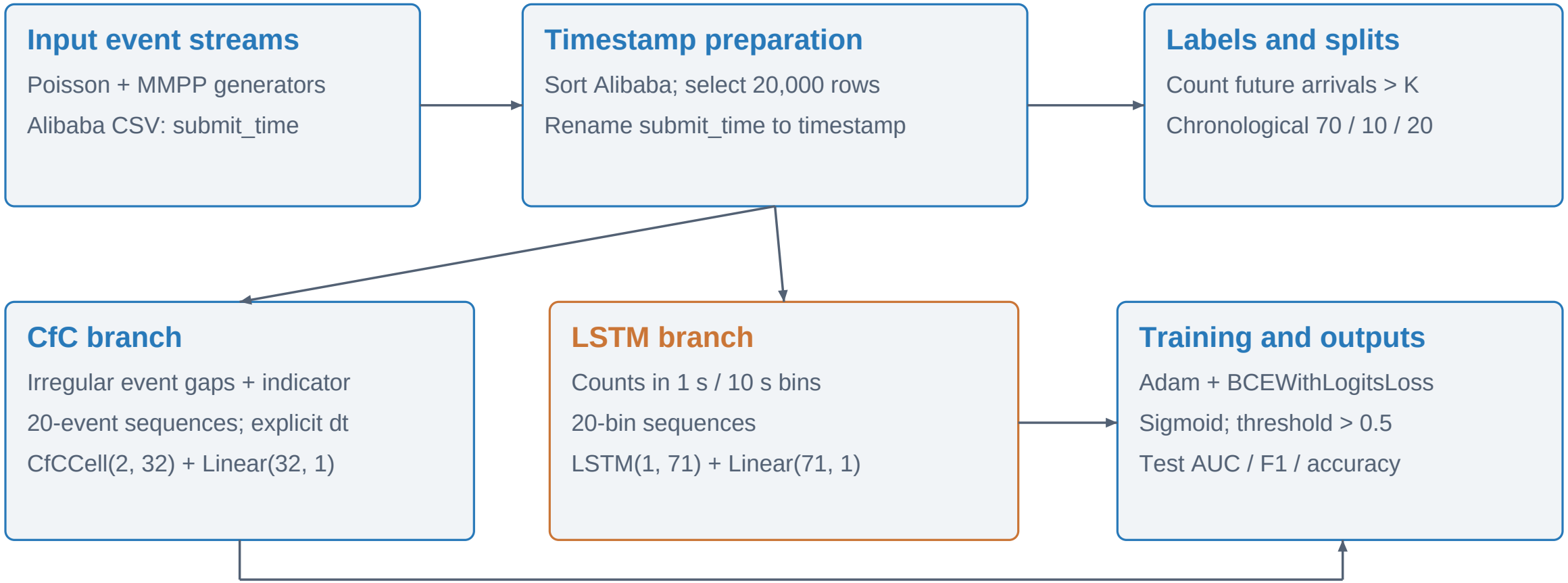
Label: y(t) = 1 if the number of arrivals in (t, t + W] is greater than K; otherwise y(t) = 0.

This labels future high-arrival windows. It does not explicitly identify the exact transition into a burst. Job-arrival bursts differ from CPU execution bursts.

Source: local project implementation and saved experiment outputs

6. Architecture: complete experiment pipeline

Implemented data paths, model branches and evaluation outputs

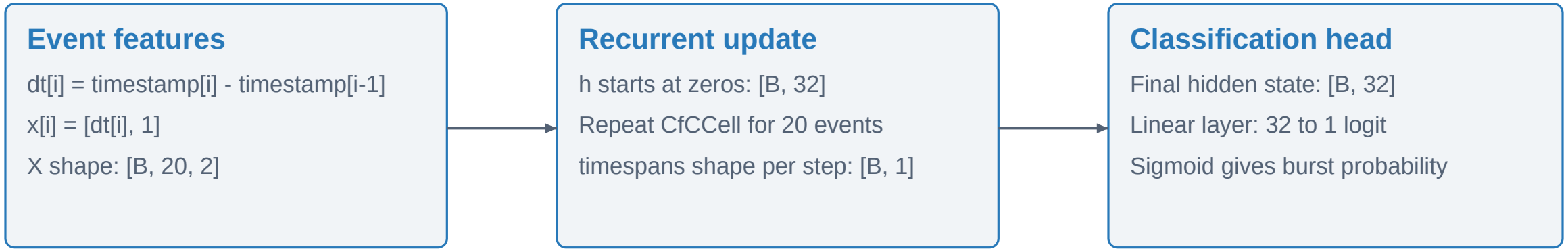


Branches train independently. Both feed the same evaluation procedure; the LSTM does not consume CfC outputs.

Source: project/main.py, data_pipeline.py and train.py

7. Architecture: CfC event processing

Closed-form Continuous-time model from ncps.torch.cfc_cell.CfCCell



Time-dependent state update

$$g = \text{sigmoid}(a * dt + b)$$

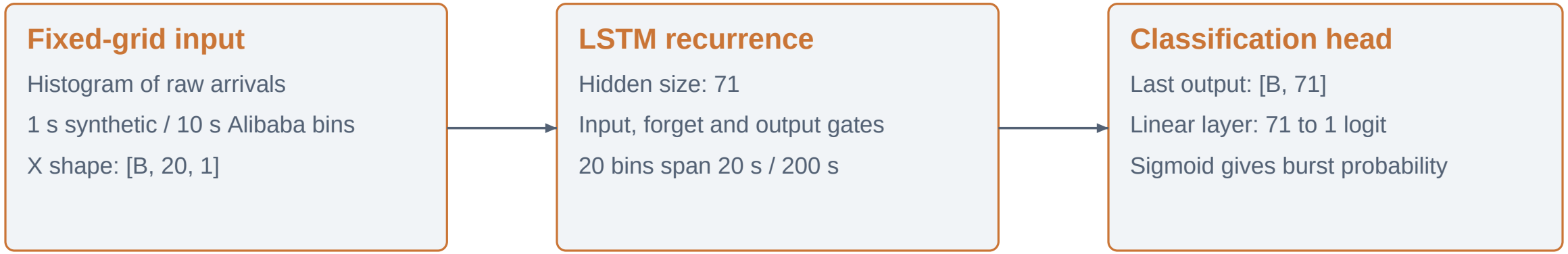
$$h_{new} = (1 - g) * u + g * v$$

The backbone processes [input, previous hidden state]. Learned projections form candidates u and v and gate terms a and b. All products above are elementwise.

dt enters both the feature vector and the timespans argument. This duplication is optional. CfC uses a direct update without a numerical ODE solver.

Source: train.py and installed ncps CfCCell default gated implementation

8. Architecture: LSTM baseline and shared target



Actual arrival counts define labels

At each bin edge, count raw future events, not grid points. The same window / threshold definition applies to both branches.

Different observation schedules

CfC predicts at event times and LSTM at bin edges. Twenty events and twenty bins cover different historical durations.

Source: data_pipeline.py and train.py

9. Training and evaluation protocol

Training configuration

20 epochs; batch size 64; Adam learning rate 0.005; binary cross-entropy with logits. Random seed 42.

Capacity

CfC: 21,025 trainable parameters, hidden size 32. LSTM: 21,088 parameters, hidden size 71.

Held-out evaluation

Chronological 70% training, 10% validation, 20% testing per pipeline. Labels use events within each split.

Metrics

Accuracy and F1 use probability > 0.5. ROC AUC uses continuous probability scores across thresholds.

Source: local project implementation and saved experiment outputs

10. Full held-out results

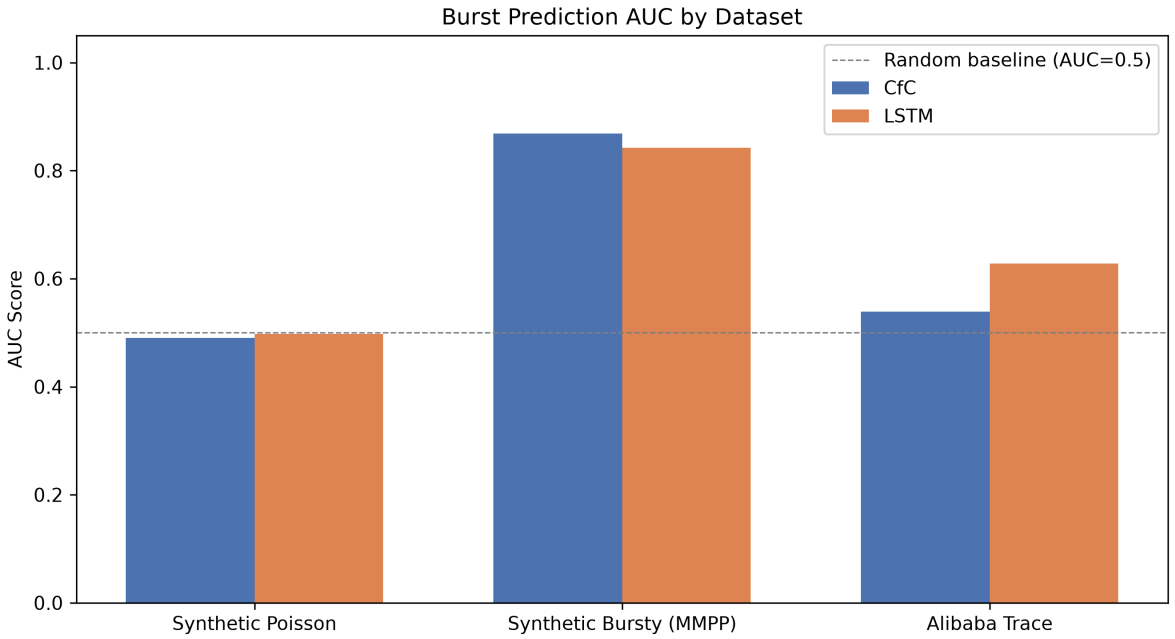
All metrics from the saved results_summary.csv; latency is milliseconds per CPU batch

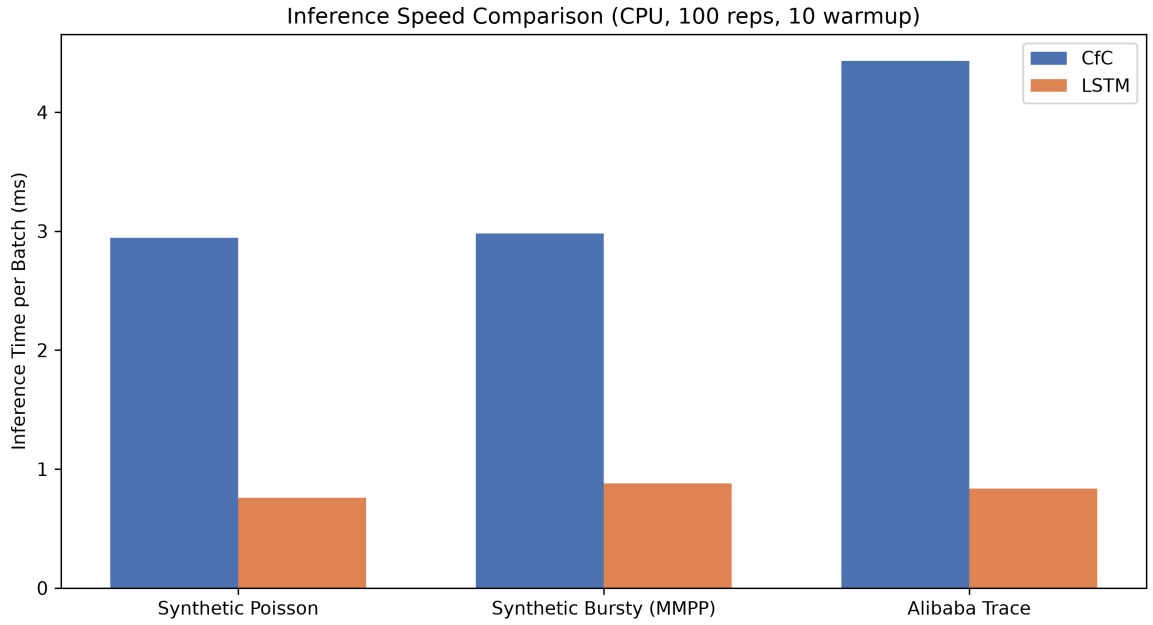
Dataset	Model	AUC	F1	Accuracy	ms / batch	Parameters
Poisson	CFC	0.4903	0.0000	0.8792	2.944	21,025
Poisson	LSTM	0.4976	0.0000	0.8769	0.759	21,088
Bursty MMPP	CFC	0.8684	0.9458	0.9163	2.982	21,025
Bursty MMPP	LSTM	0.8422	0.7432	0.9291	0.878	21,088
Alibaba PAI	CFC	0.5387	0.9843	0.9691	4.432	21,025
Alibaba PAI	LSTM	0.6278	0.9667	0.9357	0.834	21,088

MMPP AUC: CfC +0.0263. Alibaba AUC: LSTM +0.0892. These are single-run observations on different evaluation schedules.

Source: project/data/results_summary.csv; times converted from seconds to milliseconds

11. AUC and inference-time comparisons

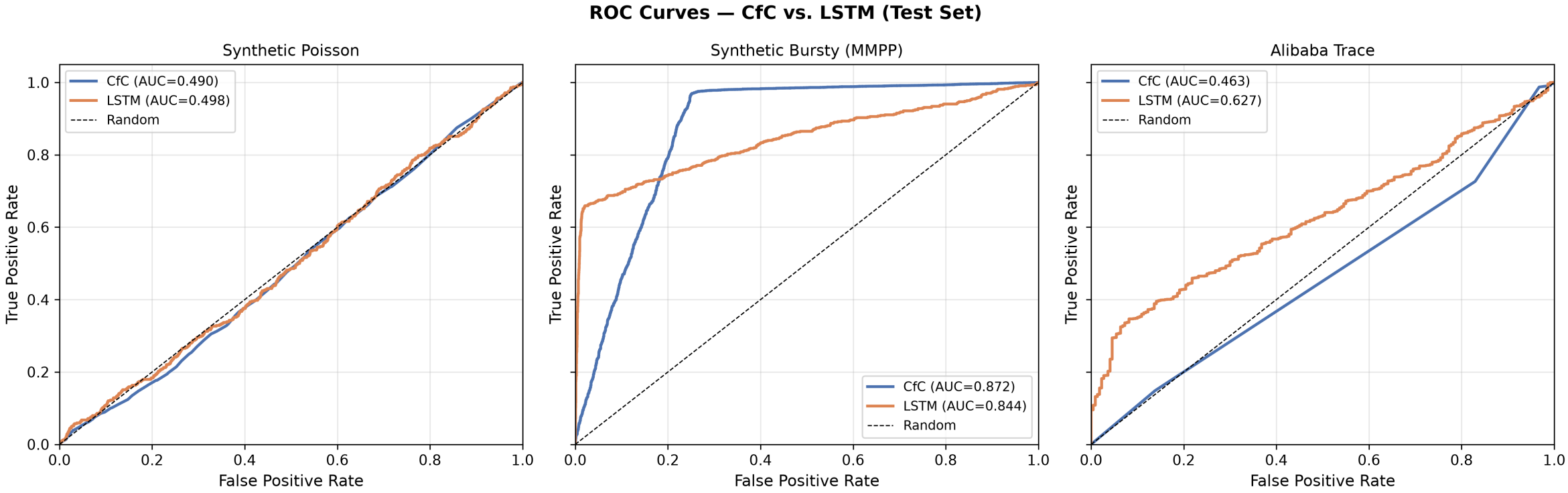




Both models rank Poisson outcomes near chance. CfC has higher MMPP AUC, while LSTM has higher Alibaba AUC. LSTM is faster in this implementation.

Source: saved project figures and results_summary.csv

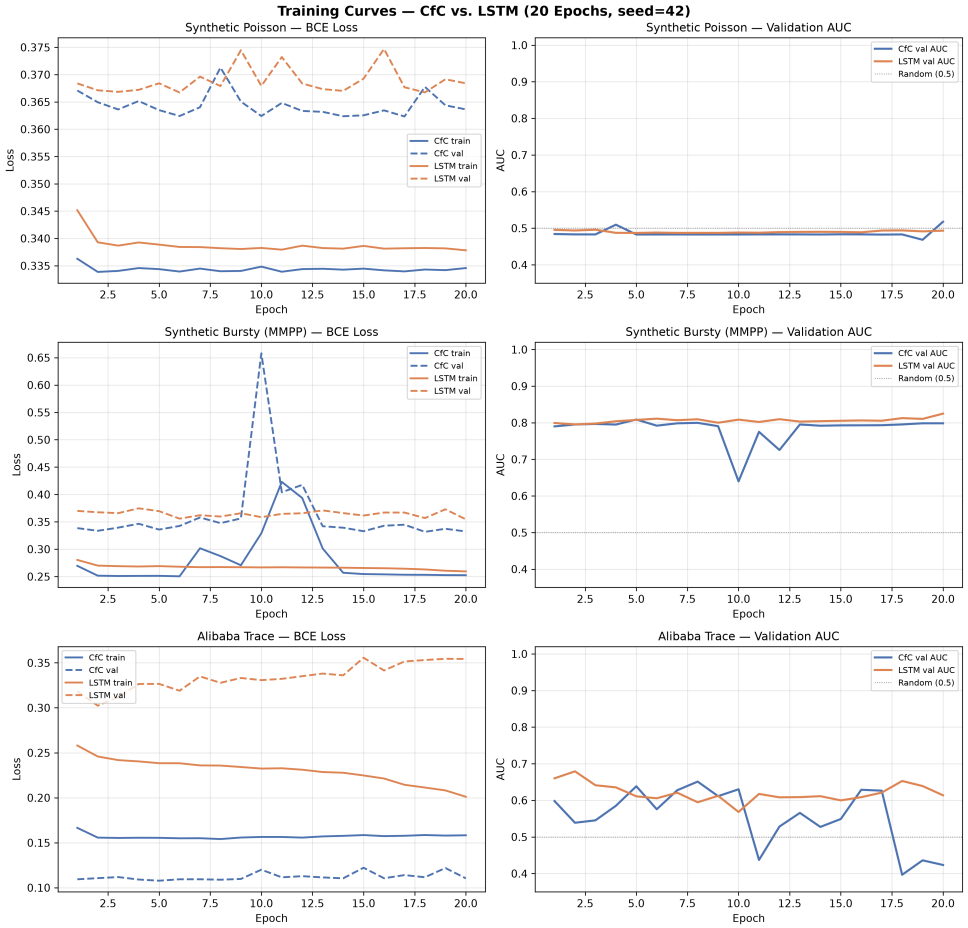
12. ROC curves and training behavior



ROC curves show ranking across thresholds. High accuracy alone does not establish discrimination, especially when burst labels dominate.

Source: project/figures/roc_curves.png; generated by generate_figures.py

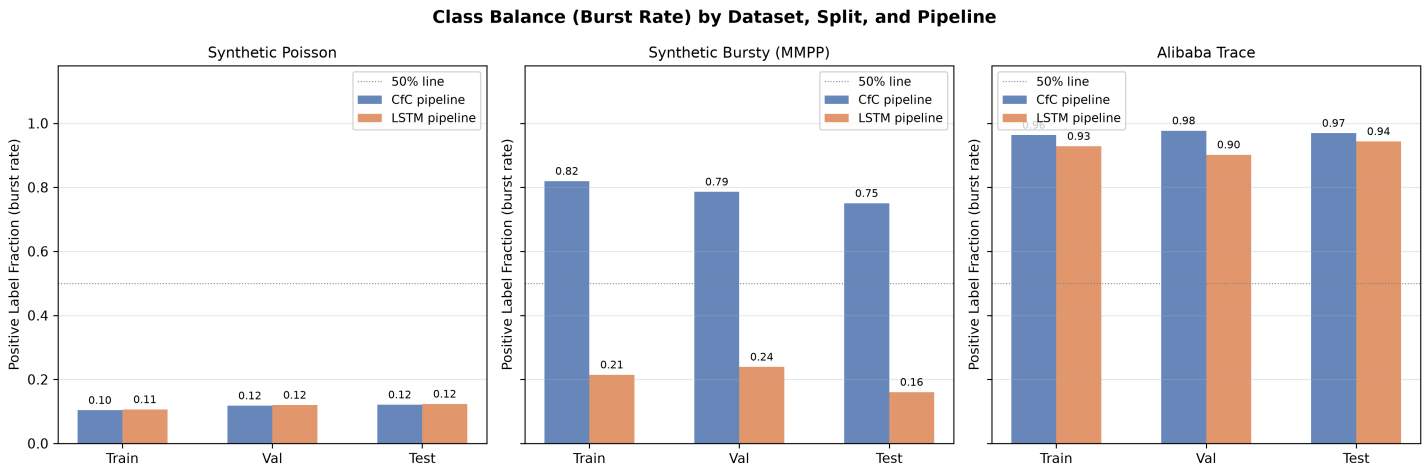
13. Training curves



Validation measurements monitor learning. The experiment evaluates final-epoch models rather than automatically selecting the best validation checkpoint.

Source: train.py and project/figures/training_curves.png

14. Class balance and interpretation



Poisson

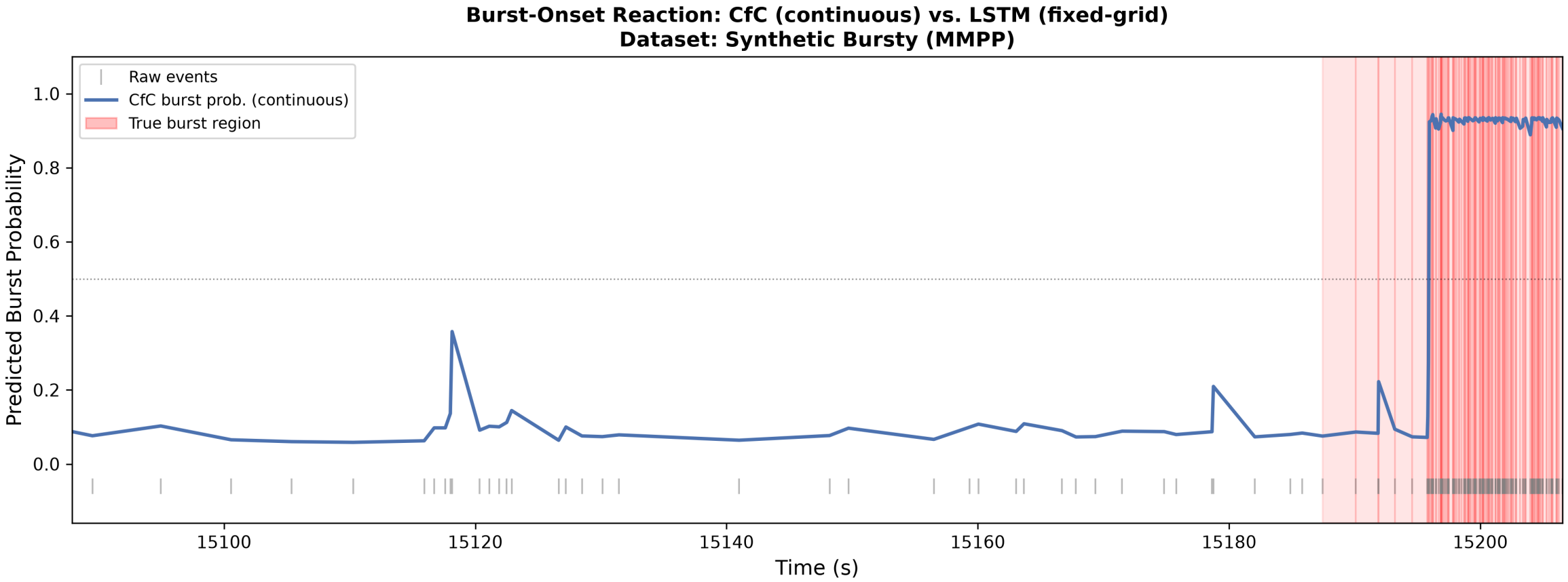
About 88% accuracy with F1 = 0 reflects majority-class predictions, not useful burst detection.

Alibaba

Burst labels dominate. CfC F1 = 0.9843 coexists with AUC = 0.5387. A high F1 can still accompany weak ranking.

Source: class_balance.png and results_summary.csv

15. Burst-reaction timeline



Illustrative event-versus-grid timing. A timeline alone does not establish a measured reduction in scheduler reaction delay.

Source: project/figures/burst_reaction_timeline.png

16. Inference benchmark and limitations

Measured CPU latency

Poisson: 2.944 / 0.759 ms; MMPP: 2.982 / 0.878 ms; Alibaba: 4.432 / 0.834 ms (CfC / LSTM).

Timing method

Mean per-batch wall-clock time over 100 repetitions, after 10 warmup passes. Batch size is up to 64 sequences.

Implementation confound

CfC executes a Python loop over steps. PyTorch LSTM uses optimized sequence processing. Timing is not a pure architecture comparison.

Deployment scope

The benchmark excludes a real scheduler and end-to-end scheduling overhead. No scheduler latency guarantee follows from these timings.

Source: local project implementation and saved experiment outputs

17. Experimental limitations and next steps

Shared evaluation needed

Align query timestamps, temporal split boundaries and history coverage before making strong model comparisons.

Label boundary handling

Exclude final windows without a full future horizon. Current split-local labels can undercount near split endings.

Uncertainty and imbalance

Repeat with multiple seeds and confidence intervals. Add PR AUC and majority / simple arrival-rate baselines.

Scheduling validation

Integrate predictions into a scheduler, then measure waiting time, queue length and resource utilization. These outcomes remain untested.

Source: local project implementation and saved experiment outputs

18. Conclusion

Controlled bursts

CfC achieved AUC 0.8684 versus LSTM 0.8422 on MMPP, with higher observed F1.

Real trace

LSTM achieved higher Alibaba AUC: 0.6278 versus CfC 0.5387. High accuracy and F1 require class-balance context.

Overall finding

Continuous-time modeling shows promise on synthetic burst structure. This experiment does not demonstrate consistent superiority or improved CPU scheduling.

Source: local project implementation and saved experiment outputs

19. Repository and reproducibility

Liquid-Neural-Networks-for-AI-Based-CPU-Scheduling

github.com/Farhan-Rashid2024/
Liquid-Neural-Networks-for-AI-Based-CPU-Scheduling

Experiment entry point

project/main.py; dataset preparation in data_pipeline.py; models and training in train.py.

Saved evidence

project/data/results_summary.csv; project/figures/; six model weight files in project/models/.

Reproduction

Install project/requirements.txt. Run from project/. Set ALIBABA_CSV or place the trace at the repository-relative dataset path.

Repository link supplied by the author; numerical evidence from the local project snapshot

QUESTIONS & DISCUSSION

Thank You